

OTIMIZAÇÃO NÃO LINEAR — TRABALHO 2

Gradiente e Matriz Hessiana

Romulo de Aguiar Beninca · Prof. Dr. Carlos Alberto Bavastri

Otimização não linear

TRABALHO 2

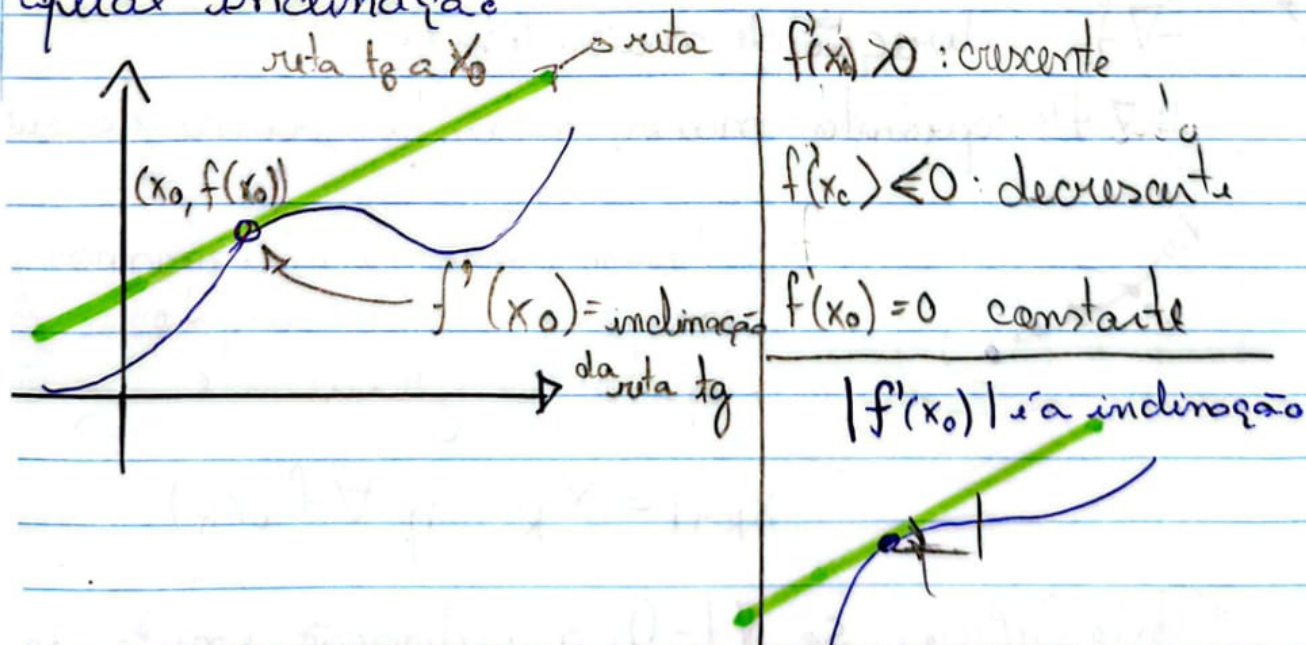
Objetivo do trabalho: Implementar em matlab duas funções que calculam por diferenças finitas, o ~~vetor~~ gradiente $\nabla f(x)$ e a matriz Hessidiana

O objetivo do projeto proposto é que o código funcione, sem depender da expressão analítica de f , ou seja, tratando f como uma função caixa preta.

Antes de implementarmos

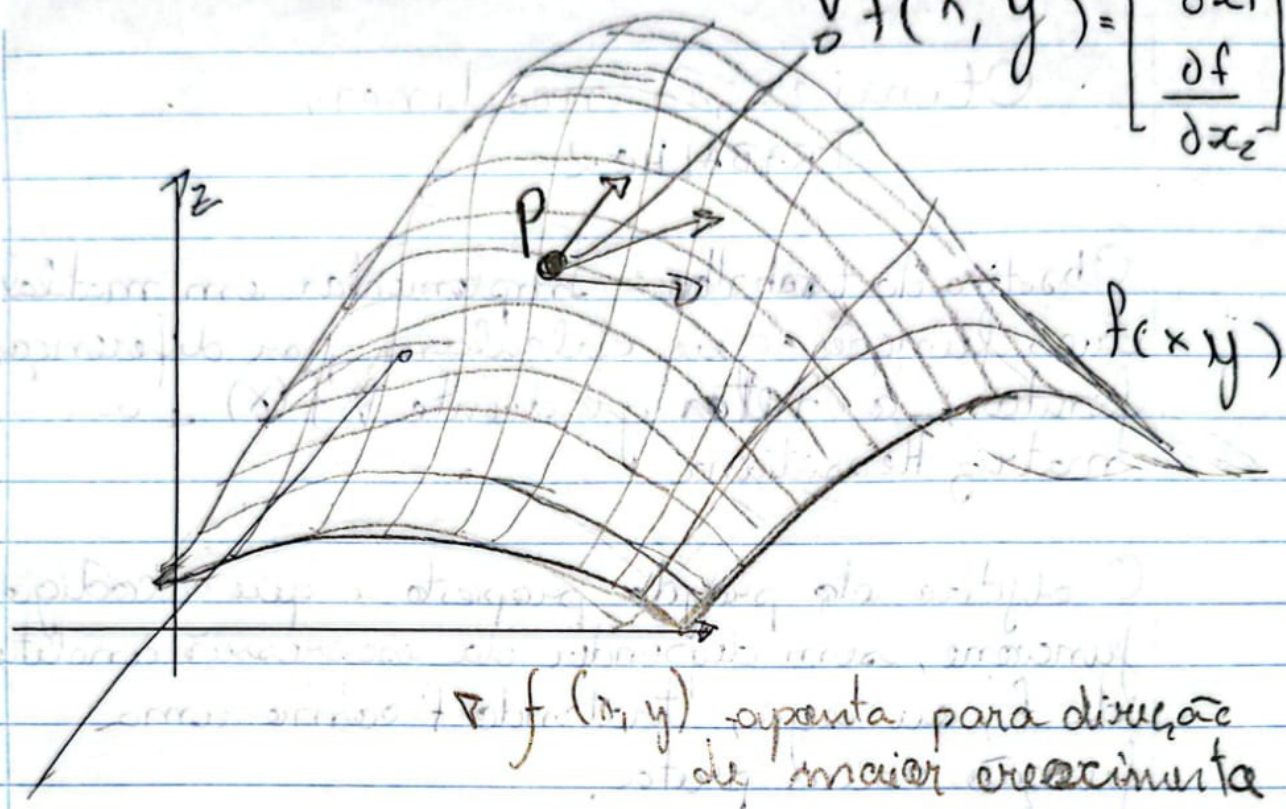
1.1) Gradiente e Hessidiana: o que são e para que servem?

Quando uma função $f(x)$ possui apenas uma variável, a sua derivada nos permite determinar para onde a função cresce e com qual inclinação



Handwritten text at the top of the page, possibly a title or header, which is mostly illegible due to blurring. It appears to contain the words "Handwritten" and "document".

$$\nabla f(x, y) = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \frac{\partial f}{\partial x_2} \end{bmatrix}$$



$\nabla f(x, y)$ aponta para direção de maior crescimento

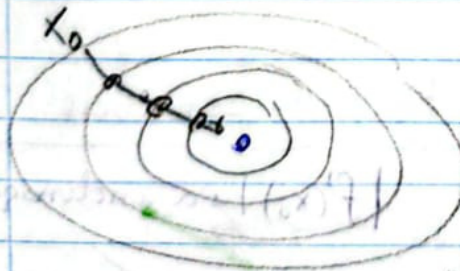
Assim o gradiente é

$$\nabla f(x_1, x_2, x_3, \dots, x_n) = \left(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \dots, \frac{\partial f}{\partial x_n} \right)$$

↳ interpretação geométrica temos que ∇f maior crescimento

$-\nabla f$ direção de maior descida

$\|\nabla f\|$ quanto maior, mais íngreme é a subida



Assim para minimizarmos f caminhamos na sentido oposto ao gradiente

$$x_{k+1} = x_k - \eta \nabla f(x_k)$$

*Ponto Crítico: se $\nabla f = 0$, a inclinação é nula em todas direções: é mínimo, máximo ou saddle point

1. The first step is to identify the problem or goal.

O gradiente, sozinho, fornece para onde a função é crescente, mas as derivadas segundas que temos se o crescimento acelera ou não, desaceleração.

Assim a Hessiana é a estrutura que organiza todas as segundas derivadas, é uma matriz quadrada ($n \times n$) que descreve a curvatura de f .

$$H_f(x) = \nabla^2 f(x) = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1 \partial x_2} & \dots & \frac{\partial^2 f}{\partial x_1 \partial x_n} \\ \frac{\partial^2 f}{\partial x_2 \partial x_1} & \frac{\partial^2 f}{\partial x_2^2} & \dots & \frac{\partial^2 f}{\partial x_2 \partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_n \partial x_1} & \frac{\partial^2 f}{\partial x_n \partial x_2} & \dots & \frac{\partial^2 f}{\partial x_n^2} \end{bmatrix}$$

3. Estrutura do código

Função	Responsabilidade
<code>principal()</code>	Script principal: define ponto e ε , chama as rotinas e imprime os resultados.
<code>funcao_objetivo(x)</code>	Avalia $f(x)$ no ponto fornecido.
<code>calcular_gradiente(ponto, epsilon)</code>	Retorna o vetor gradiente por diferença finita central.
<code>calcular_hessiana(ponto, epsilon)</code>	Retorna a matriz Hessiana derivando o gradiente numericamente.
<code>comparar_com_simbolico(ponto)</code>	Calcula gradiente/Hessiana simbólicos e mede o erro (norma) frente ao numérico.

4. Cálculo do gradiente

A ideia é simples: a função recebe um ponto e um passo pequeno (chamado de `epsilon`) e calcula a inclinação da função em cada variável, uma de cada vez. Para isso ela soma e subtrai esse passo na variável que está sendo analisada e mede o quanto a função mudou.

```
function grad = calcular_gradiente(ponto, epsilon)
    n = length(ponto);
    grad = zeros(1, n);

    for i = 1:n
        valor_original = ponto(i);

        ponto(i) = valor_original + epsilon;
        f_mais = funcao_objetivo(ponto);

        ponto(i) = valor_original - epsilon;
        f_menos = funcao_objetivo(ponto);

        grad(i) = (f_mais - f_menos) / (2 * epsilon);

        ponto(i) = valor_original; % restaura a variavel
    end
end
```

5. Cálculo da Hessiana

A Hessiana segue a mesma ideia do gradiente, só que um nível acima. Em vez de medir como a função muda, ela mede como o *gradiente* muda. Por isso, em vez de chamar `funcao_objetivo`,

esta função chama `calcular_gradiente` nos pontos para frente e para trás. O resultado é guardado em uma matriz, uma coluna para cada variável.

```
function hess = calcular_hessiana(ponto, epsilon)
    n = length(ponto);
    hess = zeros(n, n);

    for i = 1:n
        valor_original = ponto(i);

        ponto(i) = valor_original + epsilon;
        grad_mais = calcular_gradiente(ponto, epsilon);

        ponto(i) = valor_original - epsilon;
        grad_menos = calcular_gradiente(ponto, epsilon);

        hess(:, i) = (grad_mais - grad_menos)' / (2 * epsilon);

        ponto(i) = valor_original;
    end
end
```

Código completo

do arquivo hessiana.m

```
% OTIMIZAÇÃO NÃO LINEAR - TRABALHO 2
% Gradiente e Hessiana por Diferenças Finitas Centrais
%
% Programa de Pós-Graduação em Engenharia Mecânica - UFPR
% Prof. Dr. Carlos Alberto Bavastri
%
=====
====

function principal()

    ponto = [3/2, -3/2, 3/2];
    epsilon = 1e-3;

    valor_f = funcao_objetivo(ponto);

    fprintf('\n=== FUNÇÃO OBJETIVO ===\n');
```

```

fprintf('f(x) = %.6f\n', valor_f);

grad_numerico = calcular_gradiente(ponto, epsilon);

fprintf('\n=== VETOR GRADIENTE ===\n');
disp(grad_numerico);

hess_numerica = calcular_hessiana(ponto, epsilon);

fprintf('=== MATRIZ HESSIANA ===\n');
disp(hess_numerica);

fprintf('=== COMPARAÇÃO COM SOLUÇÃO SIMBÓLICA ===\n');
comparar_com_simbolico(ponto, epsilon);

```

```
end
```

```

function valor = funcao_objetivo(x)
    valor = 2*x(1)^2 + x(2) + x(3)^2 - 2*x(1);
end

```

```

function grad = calcular_gradiente(ponto, epsilon)

    n = length(ponto);
    grad = zeros(1, n);

    for i = 1:n
        ponto_mais = ponto;
        ponto_menos = ponto;

        ponto_mais(i) = ponto_mais(i) + epsilon;
        ponto_menos(i) = ponto_menos(i) - epsilon;

        f_mais = funcao_objetivo(ponto_mais);
        f_menos = funcao_objetivo(ponto_menos);

        grad(i) = (f_mais - f_menos) / (2 * epsilon);
    end
end

```

```
end
```

```

function hess = calcular_hessiana(ponto, epsilon)

    n = length(ponto);
    hess = zeros(n, n);

    for i = 1:n
        ponto_mais = ponto;
        ponto_menos = ponto;

        ponto_mais(i) = ponto_mais(i) + epsilon;
        ponto_menos(i) = ponto_menos(i) - epsilon;

        grad_mais = calcular_gradiente(ponto_mais, epsilon);
        grad_menos = calcular_gradiente(ponto_menos, epsilon);

        hess(:, i) = (grad_mais - grad_menos)' / (2 * epsilon);
    end

end

```

```

function comparar_com_simbolico(ponto, epsilon)
% função feita so para comparar se o resultado do gradiente e da
% hessiana numérica está correto com o resultado simbólico
% não foi solicitada professor, mas achei interessante fazer para
% validar o código

syms x1 x2 x3

f_sym = 2*x1^2 + x2 + x3^2 - 2*x1;

grad_sym = gradient(f_sym, [x1, x2, x3]);
hess_sym = hessian(f_sym, [x1, x2, x3]);

grad_exato = double(subs(grad_sym, [x1, x2, x3], ponto))';
hess_exata = double(subs(hess_sym, [x1, x2, x3], ponto));

grad_numerico = calcular_gradiente(ponto, epsilon);
hess_numerica = calcular_hessiana(ponto, epsilon);

erro_grad = norm(grad_numerico - grad_exato);
erro_hess = norm(hess_numerica - hess_exata, 'fro');

```

```
fprintf('\nGradiente exato:\n');  
disp(grad_exato);  
  
fprintf('Hessiana exata:\n');  
disp(hess_exata);  
  
fprintf('Erro no gradiente: %.2e\n', erro_grad);  
fprintf('Erro na Hessiana: %.2e\n', erro_hess);
```

```
end
```